

DEV_IMPL_GUIDE - FE-APP-01 BSS Backoffice Web App

1. Implementation Objective

Implement the internal Backoffice web app as the main operational surface for customer care, subscription operations, billing operations, fulfillment monitoring, work orders, stock/equipment, ticketing, reporting links, and platform operations.

2. Start Here

Read these first:

1. FE-APP-00-Frontend_Architecture_and_Shared_UX_Foundation-v1.0.md
2. FE-APP-01-BSS_Backoffice_Web_App-v1.0.md
3. DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0.md
4. EM-CFG-03 RBAC Catalog
5. DD_API-00-API_Catalog_and_Gateway_Standards-v1.0.md

3. Build Order

1. Implement shell, menu, auth guard, and role visibility.
2. Implement customer search and Customer 360 header.
3. Implement Customer 360 core panels: identity, active subscriptions, balance/debt summary, open tickets, recent timeline.
4. Add lazy-loaded panels: invoices, wallet, dunning, work orders, equipment, notifications, audit.
5. Implement ticket creation and ticket detail using TCK/ASR rules.
6. Implement subscription operation actions only when owning module APIs exist.
7. Implement order/fulfillment monitoring after FUL-02 APIs are ready.
8. Implement billing and stock operational screens in their owning module areas.
9. Add platform operations screens through approved admin proxies only.

4. Customer 360 Performance Rule

Customer 360 must not block on every module at once.

- Initial load: identity, active subscriptions, balance/debt, open tickets, recent timeline.
- Other panels lazy-load on tab open.
- Each panel has independent loading, stale, error, and retry state.
- Use a Backoffice BFF/read aggregator only for reads if direct gateway fan-out becomes slow.
- The BFF must not own business state or execute writes across modules.

5. Tests

- Customer 360 partial failure.
- Role-based menu/action visibility.
- Idempotency header on commands.
- Ticket creation from Customer 360.
- Subscription command accepted and operation status shown.
- Panel lazy loading and retry.

6. Customer 360 State Model

Build Customer 360 as a header plus independent panels:

```
type Customer360State = {
  routeCustomerId: string;
  selectedAccountId?: string;
  header: RemoteState<CustomerHeader>;
  accounts: RemoteState<CustomerAccount[]>;
  subscriptions: RemoteState<SubscriptionSummary[]>;
  balance: RemoteState<AccountBalanceSummary>;
  openTickets: RemoteState<TicketSummary[]>;
  timeline: RemoteState<TimelineItem[]>;
  lazyPanels: {
    invoices: RemoteState<InvoiceSummary[]>;
    wallet: RemoteState<WalletSummary>;
    dunning: RemoteState<DunningSummary>;
    workOrders: RemoteState<WorkOrderSummary[]>;
    equipment: RemoteState<EquipmentInstanceSummary[]>;
    notifications: RemoteState<NotificationSummary[]>;
    audit: RemoteState<AuditEvent[]>;
  };
};
```

Load order:

1. Route enters with `customer_id`.
2. Load ILM customer header and account list.
3. Select default account from ILM if the customer has one account; otherwise ask the agent to choose an account.
4. Load core panels for the selected `account_id`.
5. Lazy-load heavy panels only when opened.

Do not query billing, work order, or equipment by `customer_id` when an `account_id` is available. Customer-level search can find the customer; operational panels must be account-scoped.

7. Screen-To-API Wiring

Screen area	Owning module API	First calls to implement
Customer search	ILM-CFG-01	GET /api/customers/search?q=...
Customer header	ILM-CFG-01	GET /api/customers/{customer_id}, GET /api/customer-accounts?customerId=...
Account switcher	ILM-CFG-01	GET /api/customer-accounts?customerId=...
Subscription panel	SUB-LM-01	GET /api/subscriptions?accountId={account_id}
Subscription command buttons	SUB-WF satellites	POST /api/subscriptions/{subscription_id}/activate, /pause, /resume, /terminate, /upgrade, /downgrade, /relocate, /migrate
Restriction panel	SUB-WF-RESTRICT-01	GET /api/subscriptions/{subscription_id}/restrictions, POST /api/subscriptions/{subscription_id}/restrictions, DELETE /api/subscriptions/{subscription_id}/restrictions/{code}
Invoice panel	BIL-02-READ-01	GET /api/invoices?account_id={account_id}&limit=10
Payment panel	BIL-01-PAY-01	GET /api/payments?accountId={account_id}&limit=10
Wallet panel	BIL-05	wallet summary by account/subscription, according to BIL-05 API
Dunning panel	BIL-04	account/subscription dunning status APIs
Ticket panel	TCK-01 / ASR	GET /api/tickets?customerId=..., POST /api/tickets
Work orders	WO-01	GET /api/work-orders?accountId={account_id}
Equipment	OSR-INSTANCE-01	GET /api/equipment-instances?accountId={account_id}
Notifications	NOT-01	notification history by account/customer
Audit	EM-04 / module audit APIs	entity-scoped audit queries

8. Command Flow Example - Pause Subscription

1. Agent opens Customer 360 and selects the correct account.
2. UI loads subscriptions for account_id.
3. Agent clicks pause on subscription_id.
4. UI displays reason/date form using config from the pause DD where exposed.
5. UI submits:

```
POST /api/subscriptions/{subscription_id}/pause
Idempotency-Key: bo-pause:{subscription_id}:{formAttemptId}
Content-Type: application/json
```

```
{
  "reasonCode": "CUSTOMER_REQUEST",
  "requestedPauseDate": "2026-06-10",
  "notes": "Customer travelling for two months"
}
```

6. UI receives operation id/status from SUB-WF.
7. UI shows operation status and polls or subscribes to operation status updates.
8. UI does not manually mutate subscription status; SUB-LM state refresh confirms the result.

9. Backoffice BFF Rule

A Backoffice BFF/read aggregator is allowed only for read composition such as Customer 360. It may cache short-lived projections and normalize panel payloads. It must not own business state, execute multi-module writes, or hide which module owns each command.